

# LAB-4: VibeCoding

## Chat History Screenshots

NAME: Krishna Jha  
SRN: PES1UG24CS236

The screenshot shows the initial state of the '48\_whack-a-mole' project in VS Code. The main.py file is open, showing the game engine setup and the main loop. The Chat window on the right shows the first step of the fix: 'Fix click scoring logic in game'.

```
1 import pygame
2 from game.game_engine import GameEngine
3
4 # Initialize pygame/Start application
5 pygame.init()
6
7 # Screen dimensions
8 WIDTH, HEIGHT = 500, 500
9 SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
10 pygame.display.set_caption("Whack-a-Mole - Pygame Ver")
11
12 # Colors
13 GRASS_GREEN = (120, 170, 90)
14
15 # Clock
16 clock = pygame.time.Clock()
17 FPS = 60
18
19 # Game loop
20 engine = GameEngine(WIDTH, HEIGHT)
21
22 def main():
23     running = True
24     while running:
```

Chat: Fix click scoring logic in game

Start Over

A single click can award two points when two neighboring moles are both up and the click lands near the border between their holes. Investigate the click-hit logic and fix it so one click scores at most one point, and only when it lands directly on an active mole. Clicking an empty hole or missing entirely must not score. Leave the timer, score display, and mole pop-up timing unchanged.

Completed 7 steps in 1m 0s

Fixed the click hit logic. In `game_engine.py`, a click now scores once at most and only stops at the first hit. `hole.py` checks that the mole is active and the click falls within its visible circular area.

Focused checks passed for overlap clicks, active-mole hits, empty holes, and clicks outside the board. Timer, score display, and pop-up timing were left unchanged.

2 files changed +19 -13

The screenshot shows the progress of the '48\_whack-a-mole' project in VS Code. The main.py file is open, showing the game engine setup and the main loop. The Chat window on the right shows the second step of the fix: 'Fix click scoring logic in game'.

```
1 import pygame
2 from game.game_engine import GameEngine
3
4 # Initialize pygame/Start application
5 pygame.init()
6
7 # Screen dimensions
8 WIDTH, HEIGHT = 500, 500
9 SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
10 pygame.display.set_caption("Whack-a-Mole - Pygame Ver")
11
12 # Colors
13 GRASS_GREEN = (120, 170, 90)
14
15 # Clock
16 clock = pygame.time.Clock()
17 FPS = 60
18
19 # Game loop
20 engine = GameEngine(WIDTH, HEIGHT)
21
22 def main():
23     running = True
24     while running:
```

Chat: Fix click scoring logic in game

Restore Checkpoint

After the collision fix, add a game-over screen. When the round timer reaches zero, stop the round and show the final score on screen. Wait for input there. Do not end the round by only printing to the console. Keep scoring, the countdown, and mole behavior the same during play.

Completed 6 steps in 38s

Added a game-over screen in `game_engine.py` that displays the final score and stays visible until the window is closed. The round remains frozen at zero; scoring, countdown, and mole behavior during play are unchanged.

Verified the screen renders and that the timer, mole state, and score remain unchanged after game over.

1 file changed +17 -5

